SPRING FRAMEWORK

- **Spring is a lightweight, open-source, Java-based framework that simplifies enterprise application development.**

- **It was developed by Rod Johnson in 2003.**

- **It is mainly used to develop enterprise-level applications.**

- **It is called a "framework of frameworks" because it provides integration and support for various other frameworks such as Hibernate, JPA etc.**

- **In simple terms, the Spring framework can be defined as a comprehensive structure that provides solutions to various technical problems in application development.**

- **The Spring framework consists of several core modules, including: IOC (Inversion of Control) , Web MVC , AOP , DAO etc.**

Advantages of Spring Framework

- **Predefined Templates : Spring provides ready-to-use templates for technologies such as JDBC, Hibernate, JPA, etc. This reduces boilerplate code by hiding common steps (like opening/closing connections).**
- **Loose Coupling : Spring applications are loosely coupled due to Dependency Injection (DI). This improves flexibility and makes code easier to maintain.**
- **Easy Testing : Thanks to Dependency Injection, testing Spring applications becomes simpler.**
- **Lightweight : Spring is lightweight as it is based on POJOs (Plain Old Java Objects). It doesn't force developers to extend specific classes or implement heavy interfaces.**
- **Integration with Other Frameworks : Spring provides seamless support for integrating with other frameworks and technologies like Hibernate, JPA etc. This makes Java EE application development faster and easier.**

Inversion Of Control (IOC)

Design Patterns in Spring:

- Spring uses design pattern such as IOC , Dependency Injection (DI) to remove dependency from programming code.
- This makes applications easier to test and maintain.

IoC Container :

- It is a spring container that creates the objects , configures, assembles their dependencies and manages their entire life cycle.
- The IoC container is the core of the Spring framework.
- It uses Dependency Injection (DI) to manage application components.

** **NOTE** : The control of object creation and dependency management is given to the Spring container instead of the programmer, hence it is known as Inversion Of Control.

Responsibilities of IoC Container:

Creating objects.
Configuring objects and assembling their dependencies.
Managing the entire lifecycle of objects.

** **NOTE** : The objects created and managed by the IoC container are called Spring Beans.

Types of IoC Containers

Spring provides two main types of IoC containers:

1. **BeanFactory (Core Container)**
   - The most basic IoC container.
   - Provides configuration to instantiate, configure, and manage beans.
   - Loads bean definitions only from XML configuration.
   - Does not support annotation-based configuration.

2. **ApplicationContext (Advanced Container)**
   - Sub-interface of BeanFactory (thus supports all BeanFactory features).
   - Provides additional enterprise-specific features.
   - Supports both XML and annotation-based configurations.
   - Preferred in real-time applications because it is more powerful and feature-rich.

- **Ways to Provide Object Creation Information to IoC Container**
  - **Using XML configuration file**
  - **Using Annotations**

Steps to create a Spring core project : Using XML file

1. **Create a simple maven project and add spring context dependency.**

```
<!--https://mvnrepository.com/artifact/org.springframework/spring-context -->
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>6.1.14</version>
</dependency>
```

Example 1 :

- **Create a POJO class (class for which you want spring IOC to create object).**

```
public class Sample {

    public void test() {
        System.out.println("Non-static test() method of Sample class is executing.");
    }
}
```

- **Create an xml file and configure the bean.**

(xml file should be created directly inside src/main/java folder only.)

(xml file can have any name).

Config.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
            http://www.springframework.org/schema/beans/spring-beans.xsd
            http://www.springframework.org/schema/context
            http://www.springframework.org/schema/context/spring-context-3.0.xsd">

    <bean id="mysample" class="com.jsp.Sample"></bean>



</beans>
```

- **Create the application class to access the bean by creating IOC container (ApplicationContext) using its implementing classes such as ClassPathXmlApplicatonContext.**

```
public class SampleMain {

    public static void main(String[] args) {

        // create IOC container using ApplicationContext
            // to IOC container , provide xml file as input
        ApplicationContext ac = new ClassPathXmlApplicationContext("Config.xml");

        // from IOC container - ac , getting sample class object
        Sample s = (Sample)ac.getBean("mysample");
                    // OR
        //Sample s = ac.getBean("mysample", Sample.class);

        s.test();
    }
}
```

The xml file name in which information about bean creation is defined.

2. Create a POJO class (class for which you want spring IOC to create object).

```java
package ioc_demo_programs;

public class Student {

    private int id;
    private String name;
    private int age;
    private String address;

    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public int getAge() {
        return age;
    }
    public void setAge(int age) {
        this.age = age;
    }
    public String getAddress() {
        return address;
    }
    public void setAddress(String address) {
        this.address = address;
    }
}
```

3. **Create an xml file and configure the bean.**

(xml file should be created directly inside src/main/java folder only.)

(xml file can have any name).

configure.xml

```
http://www.springframework.org/schema/beans/spring-beans.xsd (xsi:schemaLocation) | htt
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:context="http://www.springframework.org/schema/context"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
            http://www.springframework.org/schema/beans/spring-beans.xsd
            http://www.springframework.org/schema/context
            http://www.springframework.org/schema/context/spring-context-3.0.xsd">

    <bean id="mystudent" class="ioc_demo_programs.Student"></bean>



</beans>
```

- **Create the application class to access the bean by creating IOC container (ApplicationContext) using its implementing classes such as ClassPathXmlApplicatonContext.**

```
public class Main {

    public static void main(String[] args) {

        // creating IOC container
        ApplicationContext ac = new ClassPathXmlApplicationContext("configure.xml");

        // accessing/ fetching the bean(object) from IOC container.
        Student s = (Student)ac.getBean("mystudent");

        System.out.println("Student Name : "+s.getName());
        System.out.println("Student age : "+s.getAge());
        System.out.println("Student address : "+s.getAddress());

    }
}
```

- **Dependency Injection(DI) : is the main functionality provided by IOC container.**

- **It is a design pattern that allows the spring container to inject objects into other objects during run time.**

- **The control of creating objects and managing the components is taken care by IOC container with the help of Dependency Injection.**

- **The DI emphasizes on keeping the java classes independent of each other and the container frees them from object creation and maintenance.**

- **Hence , ensuring loose coupling between classes and objects.**

Need for dependency injection ?

Suppose class One needs the object of class Two to instantiate or operate a method, then class One is said to be dependent on class Two.

Now though it might appear okay to depend on a module on the other, in the real world, this could lead to a lot of problems, including system failure. Hence such dependencies need to be avoided.

Spring IOC resolves such dependencies with Dependency Injection, which makes the code easier to test and reuse .

- **Example: Teacher and Marker**
- **Without Dependency Injection (Normal Java) –**

Here, the Teacher depends on Marker and creates it herself. If the marker changes to a whiteboard marker, you must edit the Teacher class.

```
class Teacher {
    Marker marker = new Marker();  // Teacher creates her own marker

    void teach() {
        marker.write();
    }
}
```

- **With DI (Spring Way) –**

```
class Teacher {
    Marker marker;  // Teacher needs a marker, but doesn't create it

    // Marker is given (injected) from outside
    public Teacher(Marker marker) {
        this.marker = marker;
    }

    void teach() {
        marker.write();
    }
}
```

Now, Spring will create the Marker object and give it to the Teacher. Teacher just uses it — doesn't worry about how it's made. That's Dependency Injection — dependencies (Marker) are injected from outside(from IOC).

Types of Spring Dependency Injection

There are two primary types of Spring Dependency Injection:

1. **Setter Dependency Injection (SDI)**

   Setter DI involves injecting dependencies via setter methods.

2. **Constructor Dependency Injection (CDI)**

   Constructor DI involves injecting dependencies through constructors.

3. **Variable/Field Dependency Injection**

   Variable DI involves injecting dependencies through variables directly. (not recommended – not secure)

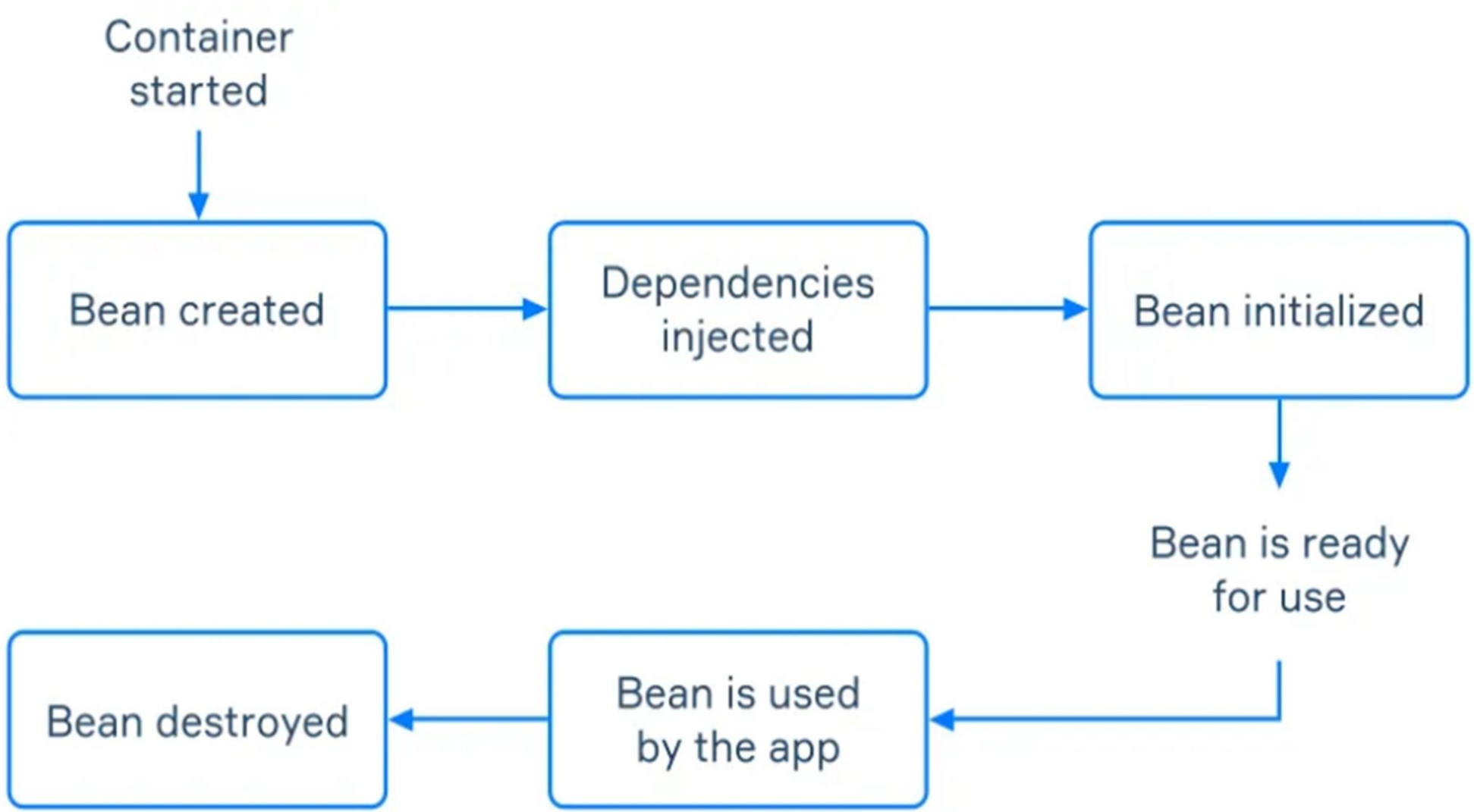
Difference Between Inversion of Control and Dependency Injection

Spring IoC (Inversion of Control)	Spring Dependency Injection
Spring IoC Container is the core of Spring Framework. It creates the objects, configures and assembles their dependencies, manages their entire life cycle.	Spring Dependency injection is a way to inject the object dependency by the following ways of spring: Constructor Injection and Setter Injection
Spring helps in creating objects, managing objects, configurations, etc. because of IoC (Inversion of Control).	Spring framework helps in the creation of loosely-coupled applications because of Dependency Injection.
IoC is a design principle where the control flow of the program is inverted(reversed).	Dependency Injection is one of the functionality of the IOC principle.

BEAN LIFE CYCLE IN JAVA SPRING

- **The bean life cycle refers to when & how the bean is instantiated, what action it performs until it lives, and when & how it is destroyed. In this article, we will discuss the life cycle of the bean.**

- **Bean life cycle is managed by the spring container. When we run the program then, first of all, the spring container gets started. After that, the container creates the instance of a bean as per the request, and then dependencies are injected. And finally, the bean is destroyed when the spring container is closed.**


Container started
Bean created
Dependencies injected
Bean initialized
Bean is ready for use
Bean is used by the app
Bean destroyed

Steps to create a Spring core project : Using Annotations

1. **Create a simple maven project and add spring context dependency.**

```
<!--https://mvnrepository.com/artifact/org.springframework/spring-context -->
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>6.1.14</version>
</dependency>
```

2. **Create a POJO class (class for which you want spring IOC to create object).**

And annotate the class with @Component

```java
@Component
public class Sample {

    public void test()
    {
        System.out.println("non static test method is executing.");
    }
}
```

3. **Create a java class and mark it as a Configuration class by using @Configuration and @ComponentScan annotations**.

```java
@Configuration
@ComponentScan(basePackages = "com.jsp")
public class Configure {

}
```

4. Create the application class to access the bean by creating IOC container (ApplicationContext) using its implementing class - AnnotationConfigApplicationContext.

```
public class SampleMain {

    public static void main(String[] args) {

        ApplicationContext ac = new AnnotationConfigApplicationContext(Configure.class);

        Sample s = (Sample)ac.getBean("sample");

        s.test();
    }
}
```

ANNOTATIONS

1. **@Component - Marks a class as a Spring-managed component. This class will be automatically detected and registered as a bean when using component scanning.**

2. **@Configuration - Marks a class as a configuration class and indicates that the class is a source of bean definitions.**

3. **@ComponentScan(basePackages = "package name") - is used to specify the base packages where Spring container should search for component classes, configuration classes etc to automatically register them as beans(to create objects for those classes). If no base package is specified , then the Spring IOC container will scan the current package and its sub packages.**

4. **@Value(value = "default value") - is used to inject default values into variables, setter methods, or constructors.**

5. **@Autowired - is used for dependency injection, allowing Spring to automatically resolve and inject the required object dependencies into a component , through a variable, method, or constructor.**

@Qualifier - When multiple beans of the same type exist, Spring throws an NoUniqueBeanDefinitionException during dependency injection because it cannot decide which bean to inject. The @Qualifier annotation allows you to specify the exact bean to use, by matching it with the bean name. It must be used along with @Autowired annotation.

@Primary - is used to specify a default bean to be injected when multiple beans of the same type are available. This simplifies dependency injection by eliminating ambiguity .It must be used along with @Component annotation on the class.

When multiple beans of the same type exist, Spring throws an NoUniqueBeanDefinitionException because it cannot decide which bean to inject. By marking one of the component class with @Primary , Spring uses that bean as the default whenever a dependency of that type is injected.

NOTE : Either @Qualifier or @Primary must be used , not both.

@Scope - are used to define the lifecycle and visibility of a bean in the application context. By default, Spring beans are singleton-scoped, meaning only one instance of the bean exists per application context. However, you can customize the bean's lifecycle.@Scope annotation must be used along with component classes.

@Scope(value="singleton") : *Default scope*: One instance per Spring Application Context.

@Scope(value="prototype") : A new bean instance is created each time it is requested

NOTE : For all the examples below , same Configure.java class is used as a configuration class.

To assign default values using @Value annotation through variables.

```java
@Component
public class Student {

    @Value(value = "Sarah")
    String name;

    @Value(value = "23")
    int age;

    @Value(value = "Bangalore")
    String address;
}
```

```java
public class StudentMain {

    public static void main(String[] args) {

        ApplicationContext ac = new AnnotationConfigApplicationContext(Configure.class);

        Student s = (Student)ac.getBean("student");

        System.out.println(s.name+" "+s.age+" "+s.address);
    }
}
```

- **To assign default values using @Value annotation through constructors.**

```
@Component
public class Employee {

    String name;
    int salary;
    String designation;

    public Employee(@Value(value = "mala") String name,@Value(value = "50000") int salary,
                                                @Value(value = "Trainee") String designation) {

        this.name = name;
        this.salary = salary;
        this.designation = designation;
    }
}
public class EmployeeMain {

    public static void main(String[] args) {

        ApplicationContext ac = new AnnotationConfigApplicationContext(Configure.class);

        Employee s = (Employee)ac.getBean("employee");

        System.out.println(s.name+" "+s.salary+" "+s.designation);
    }
}
```

- **To assign default values using @Value annotation through setter methods.**

```java
@Component
public class Person {

    private String name;
    private int age;

    public String getName() {
        return name;
    }

    @Value(value = "Ram")
    public void setName(String name) {
        this.name = name;
    }
    public int getAge() {
        return age;
    }

    @Value(value = "24")
    public void setAge(int age) {
        this.age = age;
    }


}
```

```java
public class PersonMain {

    public static void main(String[] args) {

        ApplicationContext ac = new AnnotationConfigApplicationContext(Configure.class);

        Person p = (Person)ac.getBean("person");

        System.out.println("NAME : "+p.getName());
        System.out.println("AGE : "+p.getAge());
    }
}
```

- **To inject one object inside another object (to achieve dependency injection) using @Autowired**

```java
@Component
public class Passenger {

    @Value(value = "mala")
    String name;

    @Value(value = "23")
    int age;
}
```

```java
@Component
public class Bus {

    @Value(value = "176A")
    String busno;

    @Value(value = "Garuda_Travels")
    String travelsName;

    @Autowired
    Passenger passenger;

//  @Autowired
//  public void setPassenger(Passenger passenger) {
//      this.passenger = passenger;
//  }
//
//  @Autowired
//  Bus(Passenger passenger) {
//      this.passenger = passenger;
//  }
}
```

```java
public class BusPassengerMain {

    public static void main(String[] args) {

        ApplicationContext ac = new AnnotationConfigApplicationContext(Config.class);

        Bus bus = ac.getBean("bus" , Bus.class);

        System.out.println(bus.busno+" "+bus.travelsName);
        System.out.println(bus.passenger.name+" "+bus.passenger.age);
    }
}
```

@Autowired annotation can be used with variable , setter method as well as constructor.

- **What happens when we ask IOC to inject interface type object using @Autowired ?**

```java
public interface Phone {

    void calling();
}
```

```java
@Component
public class Nokia implements Phone {

    @Override
    public void calling() {
        System.out.println("Calling from Nokia");
    }
}
```

```java
@Component
public class User {

    String name = "Suresh";

    @Autowired
    Nokia nokia;

    @Autowired
    Phone phone;
}
```

```java
public class PhoneMain {

    public static void main(String[] args) {

        ApplicationContext ac = new AnnotationConfigApplicationContext(Config.class);

        User user = ac.getBean("user" , User.class);

        System.out.println(user.name);

        user.nokia.calling();

        user.phone.calling();
    }
}
```

- **If interface is having only one implementing class , then IOC container will inject the implementing class object into interface reference.**

- **If the interface is having multiple implementing classes , then IOC container will be confused which implementing class object to inject. Hence , It throws NoUniqueBeanDefinitionException.**

• Example for usage of @Qualifier

```java
public interface Phone {

    void calling();
}
```

```java
@Component
public class Nokia implements Phone {

    @Override
    public void calling() {
        System.out.println("calling from nokia");
    }
}
```

```java
@Component
public class Nokia implements Phone {

    @Override
    public void calling() {
        System.out.println("calling from nokia");
    }
}
```

```java
@Component
public class User {

    @Autowired
    Nokia nokia;

    @Autowired
    Samsung samsung;

    @Autowired
    @Qualifier(value = "samsung")
    Phone phone;

}
```

```java
public class PhoneMain {

    public static void main(String[] args) {

        ApplicationContext ac = new AnnotationConfigApplicationContext(Configure.class);

        User u = (User)ac.getBean("user");

        u.nokia.calling();

        u.samsung.calling();

        u.phone.calling();
    }
}
```

• Example for usage of @Primary

```java
public interface Bank {

    void info();
}
```

```java
@Component
@Primary
public class HDFC implements Bank {

    @Override
    public void info() {
        System.out.println("from HDFC bank.");
    }
}
```

```java
@Component
public class SBI implements Bank{

    @Override
    public void info() {
        System.out.println("from SBI bank.");
    }
}
```

```java
@Component
public class City {

    @Autowired
    HDFC hdfc;

    @Autowired
    SBI sbi;

    @Autowired
    Bank bank;
}
```

```java
public class BankMain {

    public static void main(String[] args) {

        ApplicationContext ac = new AnnotationConfigApplicationContext(Configure.class);

        City c = (City) ac.getBean("city");

        c.hdfc.info();

        c.sbi.info();

        c.bank.info();
    }
}
```

- **Example to set the scope of bean to prototype.**

```java
@Component
@Scope(value = "prototype")
public class Flower {

}
```

```java
public class FlowerMain {

    public static void main(String[] args) {

        ApplicationContext ac = new AnnotationConfigApplicationContext(Configure.class);

        Flower f1 = (Flower)ac.getBean("flower");
        System.out.println("f1 :  "+f1);

        Flower f2 = (Flower)ac.getBean("flower");
        System.out.println("f2 :  "+f2);
    }
}
```